```
import numpy as np

marks = np.array([70, 80, 65, 90, 75])

print(marks[0])
print(marks[2])
print(marks[-1])
```

```
70
65
75
```

```
marks[0:3]
```

```
array([70, 80, 65])
```

```
marks[2:4]
```

```
array([65, 90])
```

```
marks[0:5:2]
```

```
array([70, 65, 75])
```

```
marks[:3]
```

```
array([70, 80, 65])
```

```
marks
```

```
array([70, 80, 65, 90, 75])
```

```
marks[1:]
```

```
array([80, 65, 90, 75])
```

```
marks[::]
```

```
array([70, 80, 65, 90, 75])
```

```
marks[::2]
```

```
array([70, 65, 75])
```

```
marks[1::2]
```

```
array([80, 90])
```

```
numbers = np.array([10, 20, 30, 40, 50, 60, 70])

print(numbers[1:4])
print(numbers[2:6])
print(numbers[0:3])
```

```
[20 30 40]
[30 40 50 60]
[10 20 30]
```

```
numbers = np.array([10, 20, 30, 40, 50])

print(numbers[2:])
```

```
[30 40 50]
```

```
sales = np.array([100, 150, 200, 250, 300, 350])

print(sales[-3:])
```

```
[250 300 350]
```

```
sales = np.array([100, 150, 200, 250, 300, 350])

print(sales[:-2])
```

```
[100 150 200 250]
```

```
numbers = np.array([10, 20, 30, 40, 50, 60])

print(numbers[::3])
```

```
[10 40]
```

```
numbers = np.array([10, 20, 30, 40, 50])
```

```
numbers[::-1]
```

```
array([60, 50, 40, 30, 20, 10])
```

```
marks = np.array([
    [70, 80, 90, 85],
    [60, 75, 85, 88],
    [88, 92, 95, 90],
    [65, 70, 80, 75]
])
```

```
marks[:2]
```

```
array([[70, 80, 90, 85],
       [60, 75, 85, 88]])
```

```
marks[2:4]
```

```
array([[88, 92, 95, 90],
       [65, 70, 80, 75]])
```

```
marks[1:3]
```

```
array([[60, 75, 85, 88],
       [88, 92, 95, 90]])
```

```
marks[:,:2]
```

```
array([[70, 80],
       [60, 75],
       [88, 92],
       [65, 70]])
```

```
marks[:,-2:]
```

```
array([[90, 85],
       [85, 88],
       [95, 90],
       [80, 75]])
```

```
marks
```

```
array([[70, 80, 90, 85],
       [60, 75, 85, 88],
       [88, 92, 95, 90],
       [65, 70, 80, 75]])
```

```
print(marks[1:3, 1:3])
```

```
[[75 85]
 [92 95]]
```

```
sales = np.array([
    [100, 120, 150, 180],
    [200, 220, 250, 270],
    [150, 170, 190, 210]
])

print(sales[:, 1:3] * 1.10)
```

```
[[132. 165.]
 [242. 275.]
 [187. 209.]]
```

```
prices = np.array([
    1000, 1500, 2000, 2500, 3000, 3500
])

# products 3–5 receive a 10% discount
```

```
prices[2:5] = prices[2:5] * 0.90

print(prices)
```

```
[1000 1500 1800 2250 2700 3500]
```

DPP - Practice

```
A company has recorded its monthly revenue for the first 6 months:

import numpy as np

revenue = np.array([120000, 135000, 142000, 150000, 165000, 172000])

Write a NumPy program to:

Display the revenue of the third month.
Display the revenue of the last month using negative indexing.
Select the revenue from Month 2 to Month 5 using slicing.
Increase the revenue of the last 3 months by 10%.
Display the final revenue array.
```

```
revenue = np.array([120000, 135000, 142000, 150000, 165000, 172000])
```

```
# 1. Third month
print("Third month revenue:", revenue[2])
```

```
Third month revenue: 142000
```

```
# 2. Last month
print("Last month revenue:", revenue[-1])
```

```
Last month revenue: 172000
```

```
# 3. Month 2 to Month 5
print("Month 2 to Month 5:", revenue[1:5])
```

```
Month 2 to Month 5: [135000 142000 150000 165000]
```

```
# 4. Increase last 3 months by 10%
revenue[-3:] = revenue[-3:] * 1.10
```

```
print(revenue)
```

```
[120000 135000 142000 165000 181500 189200]
```

```
A store has the following product prices:

prices = np.array([500, 750, 1200, 1500, 2000, 2500])
```

```
Write a program to:

Display the first three prices.
Display the last two prices.
Apply a 15% discount only to products at indexes 2 through 4.
Add ₹50 to the price of the first product.
Display the updated prices.
```

```
prices = np.array([500, 750, 1200, 1500, 2000, 2500])
```

```
# Display the first three prices.

prices[:3]
```

```
array([ 500,  750, 1200])
```

```
# Display the last two prices.
prices[-2:]
```

```
array([2000, 2500])
```

```
# Apply a 15% discount only to products at indexes 2 through 4.

# 3. Apply 15% discount to indexes 2, 3 and 4
prices[2:5] = prices[2:5] * 0.85
```

```
# Add ₹50 to the price of the first product.

# 4. Add ₹50 to first product
prices[0] = prices[0] + 50
```

```
print(prices)
```

```
[ 550  750 1020 1275 1700 2500]
```

```
An electricity company wants to estimate the monthly electricity consumption
of a household for 6 months.

The expected consumption is:

[180, 220, 195, 250, 280, 240] units.

The company expects consumption to increase by 8% during the next billing cycle.

Tasks
Create a NumPy array for the consumption.
Find the consumption in the 4th month.
Extract the consumption of the last 3 months.
Create a new array containing the expected consumption after an 8% increase.
```

```
consumption = np.array([180, 220, 195, 250, 280, 240])
```

```
fourth_month = consumption[3]
```

```
last_three = consumption[-3:]
```

```
predicted = consumption * 1.08
```

```
print(predicted)
```

```
[194.4 237.6 210.6 270.  302.4 259.2]
```

```
# A factory operates for 30 days and wants to create a production schedule
# starting from 500 units per day and gradually increasing to 1,500 units per day.

# Tasks
# Generate exactly 30 production targets using np.linspace().
# Create day numbers from 1 to 30 using np.arange().
# Display the first 5 production targets.
```

```
# Display the last 5 production targets.
# Increase every production target by 10%.
# Display the first 5 updated targets.
```

```
production = np.linspace(500, 1500, 30)
print(production)
```

```
[ 500.          534.48275862  568.96551724  603.44827586  637.93103448
  672.4137931   706.89655172  741.37931034  775.86206897  810.34482759
  844.82758621  879.31034483  913.79310345  948.27586207  982.75862069
 1017.24137931 1051.72413793 1086.20689655 1120.68965517 1155.17241379
 1189.65517241 1224.13793103 1258.62068966 1293.10344828 1327.5862069
 1362.06896552 1396.55172414 1431.03448276 1465.51724138 1500.        ]
```

```
days = np.arange(1, 31)
print(days)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25 26 27 28 29 30]
```

```
# First 5 targets
print("First 5 production targets:")
print(production[:5])
```

```
First 5 production targets:
[500.         534.48275862 568.96551724 603.44827586 637.93103448]
```

```
# Last 5 targets
print("\nLast 5 production targets:")
print(production[-5:])
```

```

Last 5 production targets:
[1362.06896552 1396.55172414 1431.03448276 1465.51724138 1500.        ]
```

```
# Increase production targets by 10%
updated_production = production * 1.10
```

```
print(updated_production)
```

```
[ 550.          587.93103448  625.86206897  663.79310345  701.72413793
  739.65517241  777.5862069   815.51724138  853.44827586  891.37931034
  929.31034483  967.24137931 1005.17241379 1043.10344828 1081.03448276
 1118.96551724 1156.89655172 1194.82758621 1232.75862069 1270.68965517
 1308.62068966 1346.55172414 1384.48275862 1422.4137931  1460.34482759
 1498.27586207 1536.20689655 1574.13793103 1612.06896552 1650.        ]
```

```
A college records marks of 5 students in 4 subjects.

marks = np.array([
    [72, 85, 90, 78],
    [65, 70, 75, 80],
    [88, 92, 95, 90],
    [55, 65, 60, 70],
    [78, 82, 85, 88]
])

Each row represents a student and each column represents a subject.

Tasks
Determine the number of dimensions.
Determine the number of students and subjects using shape.
Determine the total number of marks using size.
Display the complete marks of the 3rd student.
Display marks of all students in the 2nd subject.
Extract marks of the first 3 students in the first 2 subjects.
Increase the marks of the last 2 students in the 4th subject by 5.
```

```
marks = np.array([
    [72, 85, 90, 78],
    [65, 70, 75, 80],
    [88, 92, 95, 90],
    [55, 65, 60, 70],
    [78, 82, 85, 88]
```

```
])
```

```
marks.ndim
```

```
2
```

```
marks.shape
```

```
(5, 4)
```

```
# Determine the total number of marks using size.
marks.size
```

```
20
```

```
# Display the complete marks of the 3rd student.
print(marks[2])
```

```
[88 92 95 90]
```

```
# Display marks of all students in the 2nd subject.
marks[:,1]
```

```
array([85, 70, 92, 65, 82])
```

```
# Extract marks of the first 3 students in the first 2 subjects.
marks[:3, :2]
```

```
array([[72, 85],
       [65, 70],
       [88, 92]])
```

```
# Increase 4th subject marks of last 2 students
marks[-2:, 3] = marks[-2:, 3] + 5
```

```
A restaurant stores prices of 6 menu items:

prices = np.array([120, 180, 250, 300, 450, 600])

The restaurant wants to run different pricing experiments.

Tasks
Add ₹20 to the first two items.
Give a 10% discount to items 3, 4 and 5.
Keep the last item unchanged.
Display the first 3 prices after modification.
Display the last 2 prices.
Create a separate array containing only the middle three prices.
```

```
prices = np.array([120, 180, 250, 300, 450, 600])
```

```
prices[:2] += 20
```

```
prices[2:5] = prices[2:5]*0.9
```

```
prices
```

```
array([140, 200, 225, 270, 405, 600])
```

```
prices[:3]
```

```
array([140, 200, 225])
```

```
prices[-2:]
```

```
array([405, 600])
```

```
prices[2:5]
```

```
array([225, 270, 405])
```

```
A machine records temperature every hour:

temperature = np.array([
    28, 29, 30, 31, 33, 35,
    36, 35, 34, 32, 30, 29
])
Tasks
Extract readings from the first 6 hours.
Extract readings from the last 6 hours.
Extract readings from every alternate hour.
Extract readings from every third hour.
Reverse the complete temperature sequence.
Extract the last 4 readings in reverse order.
```

```
temperature = np.array([
    28, 29, 30, 31, 33, 35,
    36, 35, 34, 32, 30, 29
])
```

```
temperature.shape
```

```
(12,)
```

```
print(temperature[:6])
```

```
[28 29 30 31 33 35]
```

```
print(temperature[-6:])
```

```
[36 35 34 32 30 29]
```

```
# Extract readings from every alternate hour.
print(temperature[::2])
```

```
[28 30 33 36 34 30]
```

```
print(temperature[::3])
```

```
[28 31 36 32]
```

```
# Reverse the complete temperature sequence.

temperature[::-1]
```

```
array([29, 30, 32, 34, 35, 36, 35, 33, 31, 30, 29, 28])
```

```
temperature
```

```
array([28, 29, 30, 31, 33, 35, 36, 35, 34, 32, 30, 29])
```

```
rev_temperature = temperature[::-1]
```

```
print(rev_temperature)
```

```
[29 30 32 34 35 36 35 33 31 30 29 28]
```

```
# Extract the last 4 readings in reverse order.

print(temperature[-4:])
```

```
[34 32 30 29]
```

```
print(temperature[-4:-1:-1])
```

```
[]
```

```
print(temperature[-1:-5:-1])
```

```
[29 30 32 34]
```

HW

```
A warehouse has 4 storage sections and 5 products.

Initially, every product has a stock level of 100 units.

Tasks
Create a 4 × 5 array where every value is 100.
Increase the stock of the first two sections by 20 units.
Reduce stock of the last two products by 10%.
Display only the middle two sections.
Display only the first three products.
Display the bottom-right 2 × 2 portion of the matrix.
```

```
A company has salary data for 4 employees:

salary = np.array([
    [30000, 5000],
    [40000, 6000],
    [50000, 8000],
    [60000, 10000]
])

Column 1 = Basic Salary
Column 2 = Allowance

Tasks
Display the complete information of the 2nd employee.
Display all basic salaries.
Display all allowances.
Increase all allowances by ₹1,000.
Increase the basic salary of the last two employees by 10%.
Display the final salary matrix.
```